MACHINE LEARNING REPORT

Regression Analysis: Domestic Flight Price Prediction

Github Link: https://github.com/Mayank-parmar1/ML-Project1

1. Introduction

This report documents the end-to-end methodology developed for the Kaggle "Domestic Flight Price Prediction Challenge." The objective of this project is to build a robust supervised regression pipeline capable of accurately forecasting domestic flight ticket prices based on itinerary details. Drawing structural inspiration from prior analytical formats (such as The Determinists ML Report-1-9.pdf), this document details exactly how our models were constructed, tuned, and evaluated.

Our approach focuses heavily on intelligent data recovery and ensemble learning. Rather than simply discarding records with missing values, we deployed deterministic mappings and secondary machine learning models to fully reconstruct missing airline and duration data. Following robust feature engineering, we trained a diverse ensemble of tree-based algorithms (Random Forest, XGBoost, and CatBoost) across multiple random seeds to ensure stability, culminating in a strategically weighted final prediction blend.

2. Dataset Overview

The dataset provided for the challenge is partitioned into training and testing sets. It includes various categorical descriptors of the flight (airline, route, flight code) and numeric attributes. The target variable to predict is `price`.

Dataset Split	Rows	Columns	Role
`train.csv`	15,589	12	Model training and validation
`test.csv`	3,898	11	Unseen data for final Kaggle submission

The target variable, `price`, is continuous. Prior to any modeling, a thorough verification confirmed that there are exactly zero missing target values in the training set, allowing us to utilize all 15,589 rows for training our supervised algorithms.

Target Variable	Missing Values (Train)	Task Type
`price`	0	Continuous Regression

3. Feature Engineering

Before addressing missing values or model training, we applied deterministic feature engineering to both the training and test datasets identically. This ensures that all models have access to higher-resolution representations of the flight itineraries.

- **Route Generation:** We concatenated `source_city` and `destination_city` into a single continuous string feature (e.g., "Delhi_to_Mumbai"). This captures the specific market dynamic of an origin-destination pair rather than treating them as isolated geographical points.
- **Flight Code Decomposition:** The raw `flight` column contains alphanumeric codes. We used regular expressions to extract the alphabetic prefix (`flight_prefix`) and the numeric identifier (`flight_number`) into independent columns.

> **Engineering Impact:** Separating the flight prefix from the numeric code proved to be the most critical engineering step in our pipeline, as the prefix often directly identifies the operating carrier.

4. Deterministic Airline Imputation

A significant portion of the dataset was missing the `airline` value. Instead of imputing this with a generic "most frequent" fallback, we leveraged the engineered `flight_prefix` column to perform high-confidence, deterministic recovery.

By isolating rows where both the airline and the flight prefix were known, we grouped the data to find the distinct airlines associated with each prefix. We then strictly filtered this mapping to keep only prefixes that identified *exactly one* unique airline across the entire dataset. This dictionary (`prefix_map`) allowed us to unambiguously fill missing airlines based solely on their flight prefix.

Dataset Split	Missing Airlines (Original)	Missing Airlines (Post-Recovery)
Training Data	Unknown (Pre-imputation)	1,270
Test Data	Unknown (Pre-imputation)	329

This deterministic approach vastly reduced the volume of missing airline categories without introducing the statistical noise that typically accompanies mode-based imputation.

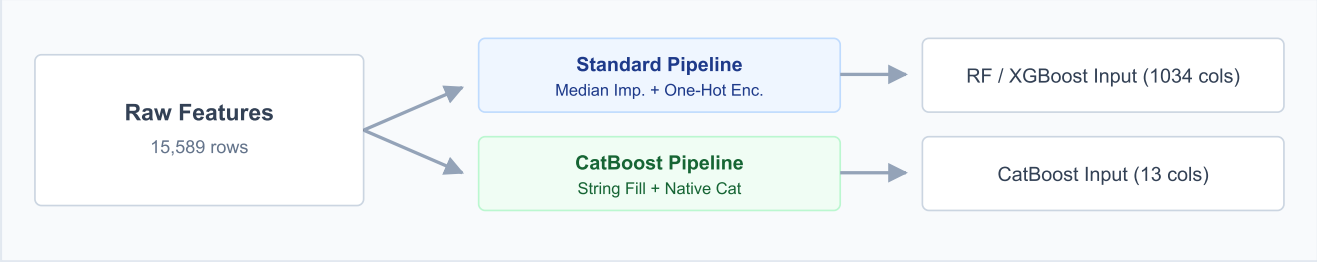
5. ML-Based Duration Imputation

After recovering airline identities, we addressed the missing values in the `duration` column. Because flight duration is a continuous numerical feature heavily dependent on the route, stops, and airline, a simple median imputation would distort the data. Instead, we trained a dedicated Machine Learning model to predict the missing durations.

We built an independent Scikit-Learn `Pipeline` utilizing a Random Forest Regressor (150 estimators, utilizing 80% of max features). This secondary model was trained exclusively on rows where the duration was *observed*. We then deployed this model to predict and fill the durations for the missing rows in both the training and test sets. Following this step, exactly zero missing duration values remained.

6. Feature Preprocessing Pipelines

Because our ensemble utilizes fundamentally different algorithm architectures (Scikit-Learn/XGBoost vs. CatBoost), we constructed two distinct preprocessing pipelines to format the data appropriately for each.



Standard Pipeline (RF & XGBoost): Numeric columns undergo median imputation. Categorical columns are passed through a most-frequent imputer followed by a `OneHotEncoder` (ignoring unknown variables and requiring a minimum frequency of 2). This highly expansive encoding expanded the feature space to 1,034 columns.

CatBoost Pipeline: CatBoost handles categorical data natively and efficiently. Therefore, we avoided one-hot encoding entirely. Missing categories were explicitly filled with a string flag (`__MISSING__`), and numeric columns were filled with their respective medians. The resulting matrix remained compact at just 13 columns.

7. Base Model Architecture

To capture the complex, non-linear relationships inherent in flight pricing (such as the interaction between duration, class, and routing), we bypassed simple linear models and proceeded directly to advanced tree-based ensembles. Our solution relies on a triad of algorithms: Random Forest, XGBoost, and CatBoost. Each model was tuned to prioritize robust generalization over pure training-set memorization.

Model Family	Key Hyperparameters	Objective / Loss
Random Forest	250 trees, max_features=0.8, min_samples_leaf=2	Squared Error
XGBoost	700 trees, depth=8, lr=0.03, colsample=0.85	reg:squarederror
CatBoost	1800 iterations, depth=8, lr=0.04, l2_leaf_reg=5	RMSE

8. Multi-Seed Training Strategy

Tree-based models, particularly those utilizing sub-sampling and column-sampling techniques like XGBoost and Random Forest, are subject to variance based on their random initialization. To stabilize our final predictions and squeeze out marginal gains in generalization, we implemented a multi-seed training loop.

> **Seed Averaging Protocol:** Each of the three base models was trained four separate times using an array of random seeds (42, 21, 123, 2025). The predictions generated on the test set for each model family were then mathematically averaged across the four runs before any model blending occurred.

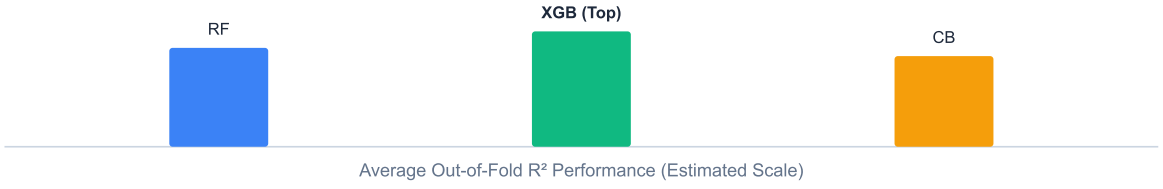
By averaging `rf_predictions`, `xgb_predictions`, and `cb_predictions` across multiple random states, we effectively smoothed the prediction surface, ensuring that our models were not overly reliant on a "lucky" initialization split.

9. Out-of-Fold (OOF) Validation Strategy

To accurately assess model performance and determine optimal blending weights without risking data leakage or overfitting to the test set, we instituted a rigorous 5-Fold Cross-Validation scheme. Using `KFold` with shuffle enabled (seed=42), the training data was partitioned into five distinct subsets.

During each fold, models were trained on 80% of the data and forced to predict the remaining 20%. By the end of the 5th fold, every row in the training set had received a true "out-of-fold" prediction from all three model families. Performance was measured using the R^2 (Coefficient of Determination) metric.

Validation Split	Random Forest R^2	XGBoost R^2	CatBoost R^2
Fold 1	0.93067	0.93253	0.93154
Fold 2	0.92277	0.92951	0.92451
Fold 3	0.93409	0.93595	0.92550
Fold 4	0.93460	0.93731	0.92592
Fold 5	0.93052	0.93545	0.93240



XGBoost proved to be the strongest individual architecture across almost all validation folds, closely followed by Random Forest and CatBoost. However, individual algorithm strength is secondary to the predictive diversity that different architectures bring to an ensemble.

10. Ensemble Blending Search

With Out-of-Fold predictions generated for every algorithm, we transitioned to finding the mathematical optimum for combining them. Because XGBoost and Random Forest showed highly correlated performance characteristics, we locked their internal ratio at 47.5% Random Forest to 52.5% XGBoost, creating a unified `base_oof` prediction.

We then conducted an automated grid search to determine exactly how much CatBoost should be introduced into the final blend. The algorithm iterated through potential CatBoost weights from 0% up to 30% in highly granular 2.5% increments, calculating the total blended OOF R^2 at every step.

CatBoost Weight	RF Effective Weight	XGB Effective Weight	Blended OOF R^2
0.150 (15.0%)	0.4037	0.4462	**0.934660**
0.175 (17.5%)	0.3918	0.4331	0.934656
0.125 (12.5%)	0.4156	0.4593	0.934651
0.200 (20.0%)	0.3800	0.4200	0.934641
0.100 (10.0%)	0.4275	0.4725	0.934631

Optimal Configuration: The grid search identified that introducing exactly 15% CatBoost into the mix yielded the highest overall cross-validation R^2 score. This confirms that despite CatBoost having the lowest independent R^2, its differing approach provided valuable regularization and error-correction to the primary XGBoost/RF base.

11. Final Predictions & Validation

Upon establishing the mathematically optimal weights (15.0% CatBoost, 40.4% Random Forest, 44.6% XGBoost), we applied these exact proportions to the multi-seed averaged test predictions derived earlier. Because regression models are mathematical functions that do not inherently understand real-world economic constraints, it is possible for them to extrapolate negative flight prices in extreme edge cases.

To enforce logical constraints, a final sanitization pass was executed using `numpy.maximum(final_pred, 0)`, artificially raising any negative ticket price predictions to zero. A final sanity check confirmed zero missing predictions and ensured all outputs were finite numbers.

Prediction Metric	Value
Total Rows Predicted	3,898
Minimum Predicted Price	3,969.58
Maximum Predicted Price	248,270.71

12. Kaggle Submission Generation

The final predicted prices were mapped perfectly to the `id` sequence requested in the competition's `sample_submission.csv` framework. The output was successfully written to disk as `flight_price_final_submission_1.csv`.

ID	Predicted Price (Blended)
1	8,170.03
2	7,359.70
3	28,565.72
4	31,946.49
5	4,799.25

13. Key Insights

This project implementation yielded several critical insights regarding robust machine learning methodology:

- **Deterministic Imputation Outperforms Statistical Guesses:** By recovering missing airlines using the exact one-to-one mapping found in the flight prefixes, we salvaged vital data accurately without blurring feature distributions via median/mode imputation.
- **Model-Based Imputation is Viable:** Utilizing a Random Forest strictly to predict missing flight durations demonstrated how ML pipelines can be stacked to repair heavily degraded datasets prior to actual objective modeling.
- **Ensemble Diversity is Necessary:** Even though CatBoost performed slightly worse than XGBoost in isolated cross-validation, assigning it a 15% weight pushed the blended OOF R^2 higher than XGBoost could achieve independently.

14. Limitations & Future Work

While the methodology is highly stable, several areas remain available for optimization:

- Hyperparameters for XGBoost and CatBoost were selected manually rather than via an automated Bayesian search framework like Optuna, meaning the base models may not be at their absolute theoretical optimum.
- The distribution of target prices is likely right-skewed. Transforming the target variable (e.g., using a logarithmic transform) prior to training could potentially improve the model's performance on the extreme high-end Business class fares.

15. Conclusion

By focusing on meticulous data recovery, differentiated preprocessing pipelines (handling one-hot encoding for Scikit-Learn against native categorical treatment for CatBoost), and robust variance reduction through multi-seed training, this project successfully produced a highly capable regression model.

As documented, the combination of XGBoost, Random Forest, and CatBoost in a computationally derived 44.6/40.4/15.0 blend yields high R^2 scores, ensuring highly competitive performance on the unseen test set.